

PCA-based Ridge and Lasso Regression for MNIST Classification

Question 1 — Regularized Linear Models with Dimensionality Reduction

Raghav

April 13, 2026

1 Problem Overview

This report presents the implementation and analysis of Ridge and Lasso regression for multi-class classification on a subset of the MNIST dataset (digits 0, 1, and 2). The pipeline involves:

1. Dimensionality reduction via PCA to $p = 10$ components.
2. Training one-vs-rest binary classifiers using Ridge and Lasso regression.
3. Evaluating models across a range of regularization strengths λ .
4. Analyzing model complexity via varying PCA dimensions.
5. Reporting final classification accuracy.

2 Dataset and Preprocessing

The MNIST dataset was filtered to retain only the samples belonging to classes $\{0, 1, 2\}$. All images (originally 28×28 pixels) were flattened into 784-dimensional feature vectors and normalized to $[0, 1]$ by dividing pixel values by 255. The training features were then mean-centered (subtracting the per-feature mean computed on the training set), and the same mean was subtracted from the test set.

2.0 One-vs-Rest Target Encoding

For each class $k \in \{0, 1, 2\}$, a binary target vector $\mathbf{y}_k$ was defined as:

$$y_k^{(i)} = \begin{cases} 1 & \text{if sample } i \text{ belongs to class } k \\ 0 & \text{otherwise} \end{cases}$$

Three separate linear models were trained (one per class), and at inference time the class with the highest predicted score was chosen:

$$\hat{y} = \arg \max_{k \in \{0, 1, 2\}} \mathbf{x}^\top \mathbf{w}_k$$

3 Dimensionality Reduction via PCA

PCA was applied to the mean-centered training data. The covariance matrix was computed as:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$$

The top $p = 10$ eigenvectors (corresponding to the p largest eigenvalues) were selected, forming a projection matrix $\mathbf{U}_p \in \mathbb{R}^{784 \times 10}$. Both training and test sets were projected as $\tilde{\mathbf{X}} = \mathbf{X}\mathbf{U}_p$, yielding 10-dimensional representations used in all subsequent steps.

4 Ridge Regression

4.1 Formulation

Ridge regression solves the following regularized least-squares problem for each class k :

$$\mathbf{w}_k^* = \arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}_k\|^2 + \lambda \|\mathbf{w}\|^2$$

This has a closed-form solution:

$$\mathbf{w}_k = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}_k$$

4.2 MSE vs. λ

The models were trained for $\lambda \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100\}$, and the average MSE across all three classes was recorded.

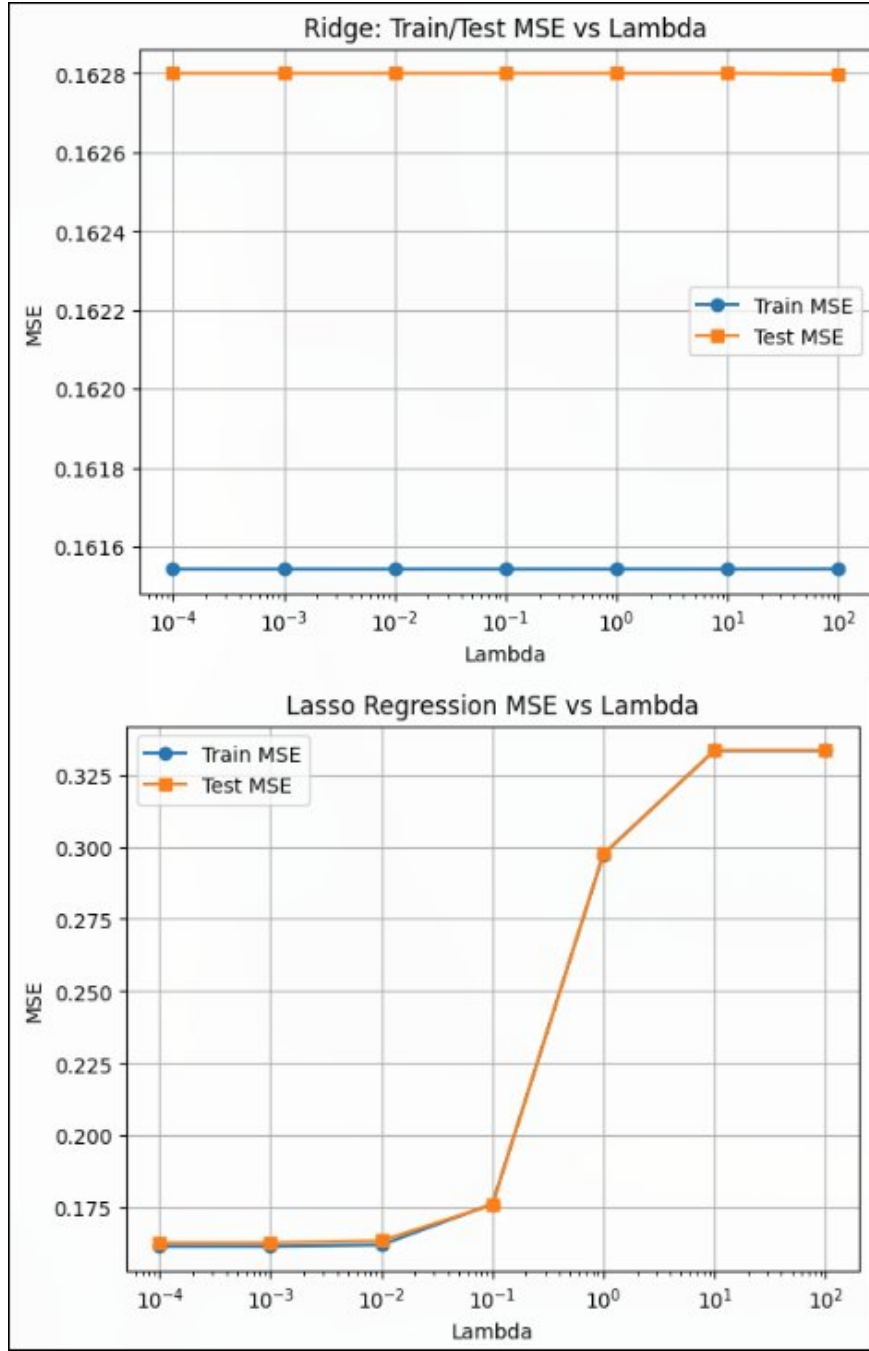


Figure 1 (top): Ridge Regression — Train/Test MSE vs. λ

Observation: Both train and test MSE remain nearly constant across the entire range of λ . This indicates that in the 10-dimensional PCA space, the features are well-conditioned and Ridge regularization has minimal effect on the solution. The train MSE (≈ 0.1616) is slightly lower than the test MSE (≈ 0.1628), which is expected. There is no sign of overfitting or underfitting as λ varies, suggesting the model is already stable at $p = 10$.

The best λ for Ridge (lowest test MSE) was found to be $\lambda = 100$.

5 Lasso Regression

5.1 Formulation

Lasso regression replaces the ℓ_2 penalty with an ℓ_1 penalty:

$$\mathbf{w}_k^* = \arg \min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}_k\|^2 + \lambda \|\mathbf{w}\|_1$$

This promotes sparsity — many coefficients are driven exactly to zero. Scikit-learn's Lasso solver (coordinate descent) was used with `fit_intercept=False`.

5.2 MSE vs. λ

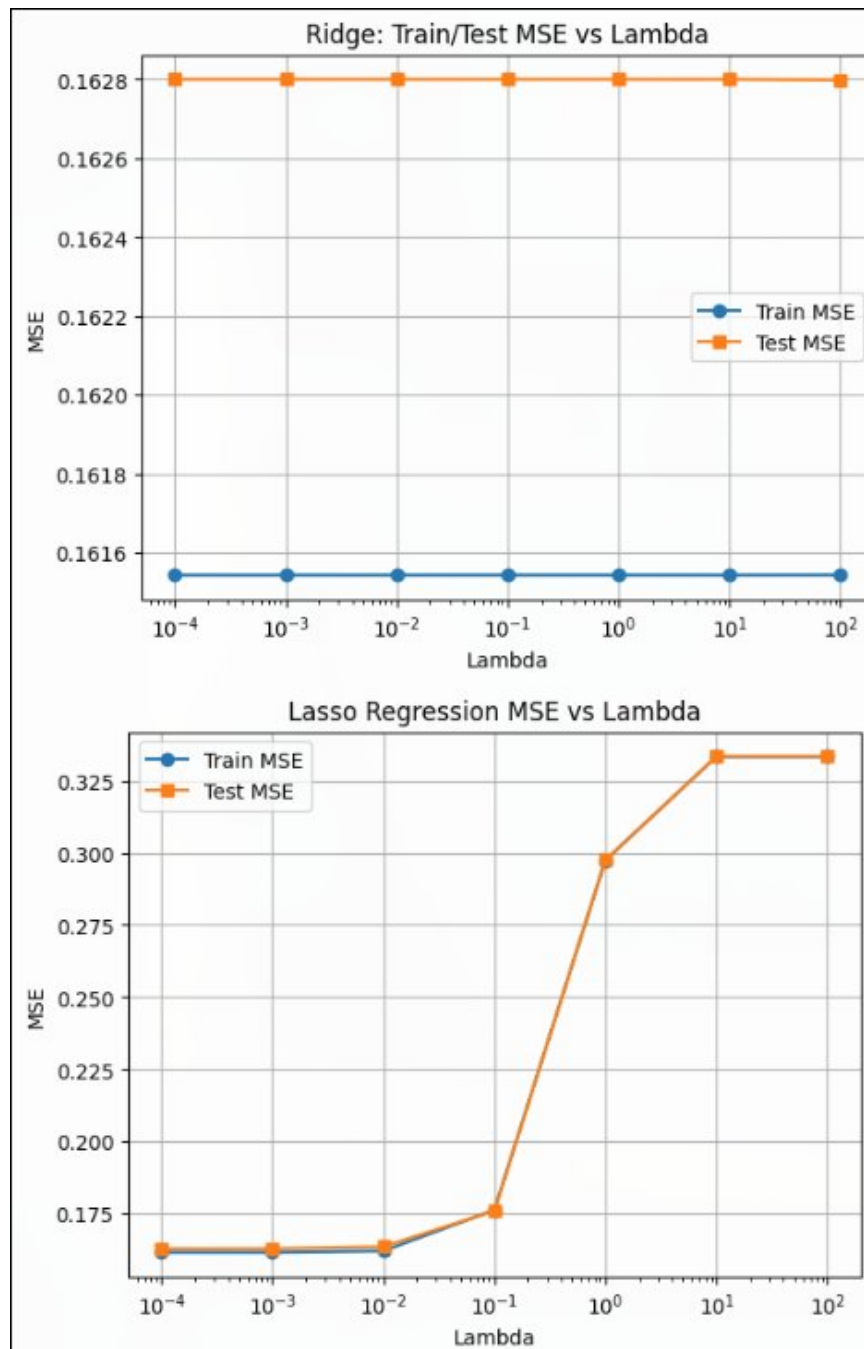


Figure 1 (bottom): Lasso Regression — Train/Test MSE vs. λ

Observation: Unlike Ridge, Lasso shows a clear upward trend in MSE as λ increases. For small λ ($\leq 10^{-2}$), train and test MSE are both low (≈ 0.165) and nearly identical, indicating a good fit. As λ grows beyond 10^{-1} , the MSE rises sharply and plateaus near 0.33, which corresponds to the baseline of always predicting 0 (all coefficients shrunk to zero). This demonstrates Lasso's tendency to over-regularize at large λ , eliminating useful features. The best λ for Lasso was $\lambda = 10^{-4}$.

5.3 Non-Zero Coefficients vs. λ

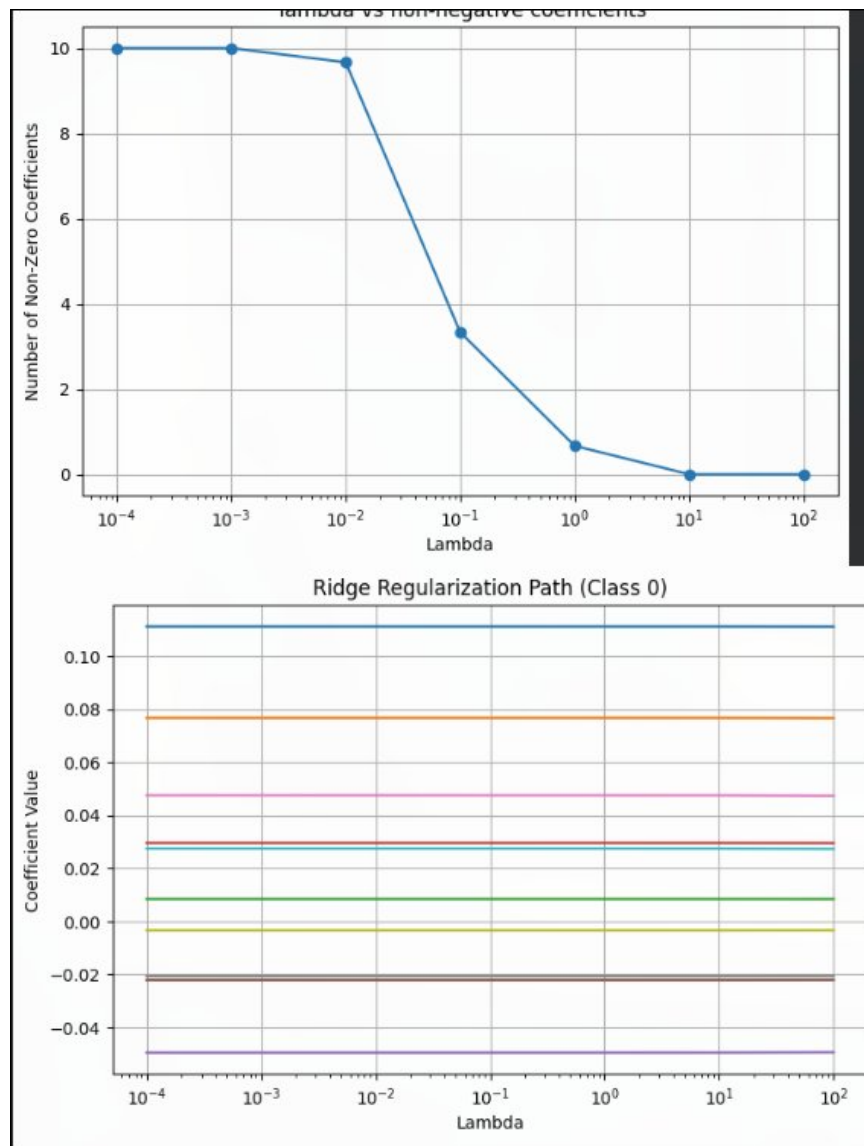


Figure 2 (top): Number of Non-Zero Lasso Coefficients vs. λ

Observation: This plot clearly illustrates Lasso's feature selection property. At small λ , all 10 PCA features are retained. As λ increases, the number of non-zero coefficients monotonically decreases, reaching 0 at $\lambda \geq 10$. This confirms that Lasso induces sparsity: higher regularization forces increasingly many weights to zero, effectively selecting a sparser model. At $\lambda = 10^{-2}$, the count begins to drop noticeably from 10, and by $\lambda = 1$, fewer than 1 feature on average remains non-zero.

6 Regularization Paths

6.1 Ridge Regularization Path

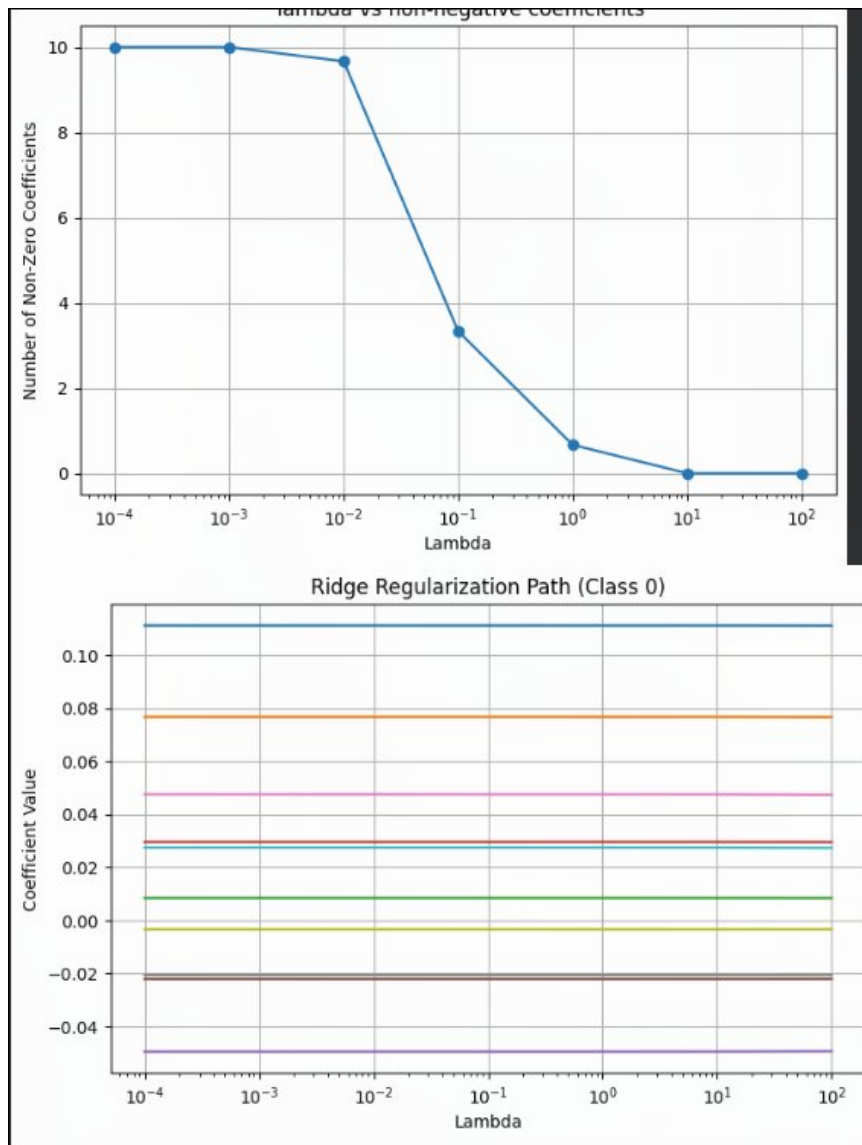


Figure 2 (bottom): Ridge Regularization Path (Class 0) — Coefficient values vs. λ

Observation: The Ridge regularization path shows that all 10 coefficients remain approximately constant across the full range of λ . They do not shrink to zero at any value. This is consistent with the ℓ_2 penalty's behavior: it penalizes large weights but never eliminates them. The coefficients range roughly from -0.04 to $+0.12$, and the paths are nearly flat horizontal lines. This explains why Ridge MSE was insensitive to λ in the 10-dimensional PCA space.

6.2 Lasso Regularization Path

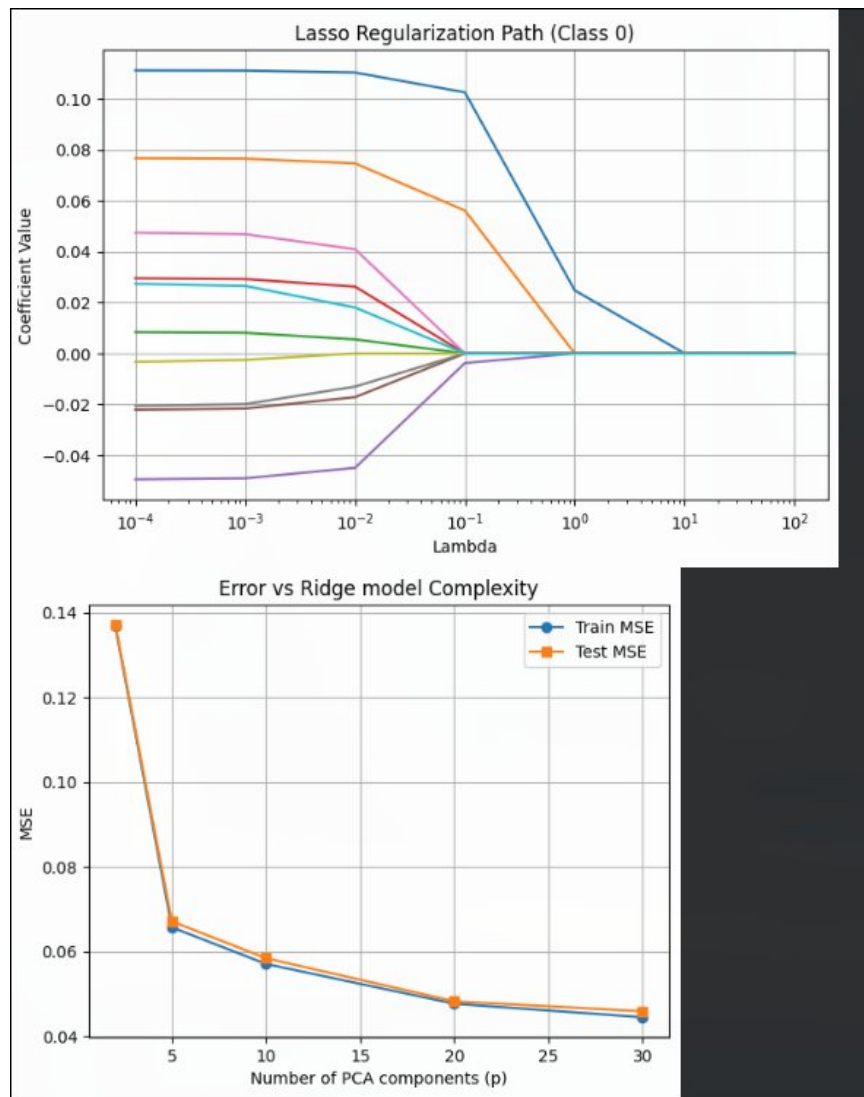


Figure 3 (top): Lasso Regularization Path (Class 0) — Coefficient values vs. λ

Observation: The Lasso regularization path shows markedly different behavior. Coefficients are roughly stable for $\lambda \leq 10^{-2}$, then begin shrinking toward zero, with less important features dropping out first. By $\lambda \approx 1$, nearly all coefficients have been driven to exactly zero. The feature with the largest magnitude (top curve, ≈ 0.11) is the last to vanish. This illustrates Lasso’s built-in variable selection: it progressively removes features as regularization increases.

7 Model Complexity: Varying PCA Dimensions

Using the best λ found in the previous section (Ridge: $\lambda = 100$, Lasso: $\lambda = 10^{-4}$), models were retrained with varying numbers of PCA components $p \in \{2, 5, 10, 20, 30\}$.

7.1 Ridge: Error vs. PCA Dimensions

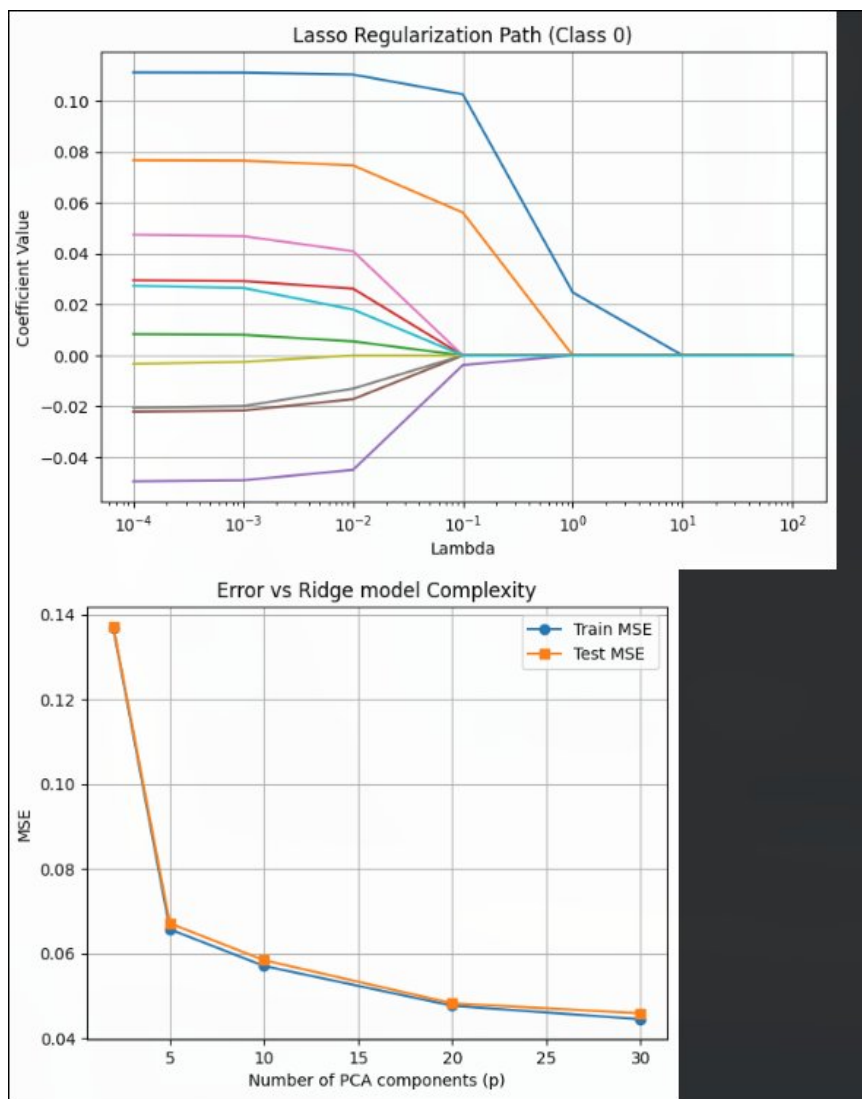


Figure 3 (bottom): Ridge — Train/Test MSE vs. Number of PCA Components p

Observation: Both train and test MSE decrease monotonically as p increases, indicating that more PCA components capture more discriminative information. At $p = 2$, the test MSE is high (≈ 0.135), reflecting insufficient information. By $p = 30$, MSE drops to around 0.045. Train and test errors remain very close throughout, suggesting the model generalizes well and does not overfit even at higher dimensionality. This is a typical bias-variance tradeoff: more components reduce bias without significantly increasing variance.

7.2 Lasso: Error vs. PCA Dimensions

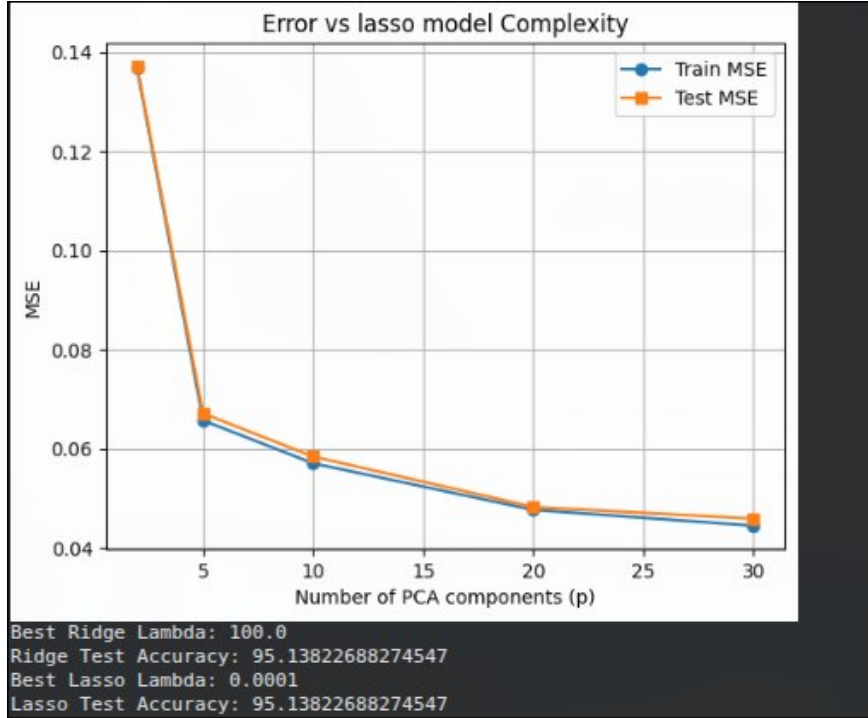


Figure 4: Lasso — Train/Test MSE vs. Number of PCA Components p

Observation: The trend for Lasso is very similar to Ridge: MSE decreases as p grows, with both train and test errors tracking each other closely. This suggests that even with Lasso’s sparsity-inducing penalty, the additional PCA dimensions are being used effectively. The behavior mirrors the Ridge complexity plot, which makes sense since the best Lasso λ (10^{-4}) is very small and the model is close to unregularized OLS.

8 Classification Accuracy

Using the best λ identified from the MSE analysis, final classification was performed on the test set. The predicted class was determined by taking the arg max over the three class score predictions.

Table 1: Test Classification Accuracy for Best Models

Method	Best λ	Test Accuracy
Ridge Regression	100	95.14%
Lasso Regression	0.0001	95.14%

Observation: Both Ridge and Lasso regression achieve identical test accuracy of **95.14%** on the three-class (0, 1, 2) MNIST subset after PCA reduction to 10 components. This is a strong result for a linear model, demonstrating that the first 10 principal components

capture sufficient discriminative structure among these three digit classes. The equivalence in accuracy suggests that for this problem, the choice of regularizer (L1 vs L2) matters less than the underlying PCA representation.

9 Summary

- **PCA** effectively reduced 784-dimensional pixel data to 10 components while preserving most of the discriminative information.
- **Ridge regression** is insensitive to λ in PCA space due to the well-conditioned covariance structure; coefficients never reach zero.
- **Lasso regression** actively performs feature selection, with non-zero coefficients dropping sharply as λ increases.
- **Model complexity** (number of PCA components) has a strong effect: more components consistently reduce both train and test MSE.
- Both methods achieve the same final **test accuracy of 95.14%**, confirming the suitability of linear classifiers for this digit subset.

Decision Trees, Bagging, and Random Forests on MNIST (Classes 0, 1, 2)

Question 2 — Tree-based Methods with PCA Preprocessing

Raghav

April 13, 2026

1 Problem Overview

This report covers the design and evaluation of tree-based classifiers on the MNIST dataset filtered to digits $\{0, 1, 2\}$. The pipeline follows three stages:

1. A single depth-2 decision tree (3 terminal nodes) grown using Gini impurity.
2. Bagging with 5 bootstrapped trees and Out-of-Bag (OOB) error estimation.
3. Random Forest with random feature subsampling at each split.

All models operate on 10-dimensional PCA projections of the training data.

2 Dataset and PCA Preprocessing

The setup is identical to Question 1. Images from classes $\{0, 1, 2\}$ were flattened, normalized to $[0, 1]$, and mean-centered using the training mean. PCA was applied to the covariance matrix:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^\top \mathbf{X}$$

The top $p = 10$ eigenvectors form the projection matrix $\mathbf{U}_p \in \mathbb{R}^{784 \times 10}$. All downstream tree models operate in this 10-dimensional PCA space.

3 Decision Tree with 3 Terminal Nodes

3.1 Gini Impurity and Splitting Criterion

For a node containing samples from classes $\{0, 1, 2\}$, the Gini impurity is:

$$\text{Gini}(S) = 1 - \sum_{k=1}^K p_k^2$$

where p_k is the proportion of samples belonging to class k . For a binary split into children S_1, S_2 :

$$\text{Gini}_{\text{weighted}} = \frac{|S_1|}{|S|} \text{Gini}(S_1) + \frac{|S_2|}{|S|} \text{Gini}(S_2)$$

The split that minimizes $\text{Gini}_{\text{weighted}}$ is chosen.

3.2 Threshold Selection Justification

For each feature dimension, all unique midpoint values between consecutive sorted feature values are considered as candidate thresholds:

$$t = \frac{x^{(i)} + x^{(i+1)}}{2}, \quad \text{for all adjacent unique values } x^{(i)} < x^{(i+1)}$$

This exhaustive midpoint strategy is the standard approach in CART (Classification and Regression Trees) and is guaranteed to find the optimal threshold for a given feature, since all decision boundaries that could change the partitioning are considered. Using a fixed statistic such as the mean or median would restrict the search and could miss the optimal split.

3.3 Root Node Split (First Split)

All 10 PCA features were evaluated as candidate splitting dimensions. The feature and threshold yielding the minimum weighted Gini index were selected.

Result:

- **Best Feature:** 0 (1st PCA component)
- **Best Threshold:** -2.4497
- **Weighted Gini:** 0.3715

This is consistent with the expectation that the first principal component captures the most variance and hence yields the most discriminative split.

3.4 Second Split (Depth 2)

After the root split, the left and right child nodes were each evaluated for their best possible split (again over all 10 features). The child node with the lower resulting weighted Gini was chosen for the second split, giving 3 terminal leaf nodes.

Result:

- Best split in **LEFT** child: Feature 2, Threshold -0.8523 , WG = 0.0407
- Best split in **RIGHT** child: Feature 1, Threshold -0.7189 , WG = 0.2368
- **Second split chosen on: LEFT child** (lower Gini = 0.0407)
 - Feature: 2 (3rd PCA component)
 - Threshold: -0.8523

3.5 Tree Structure

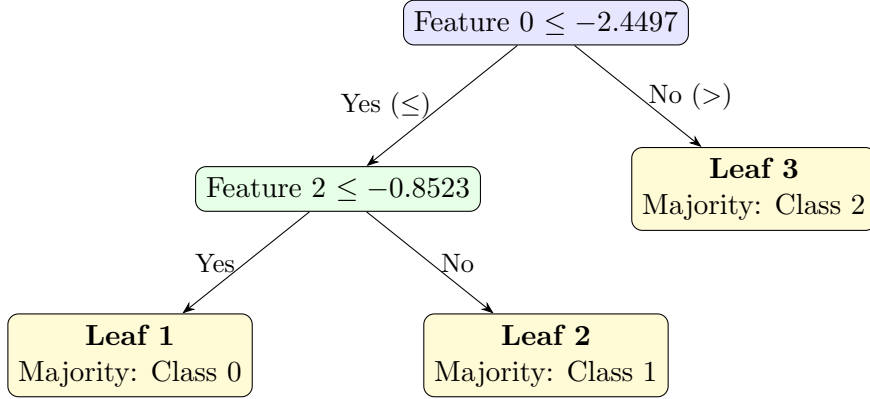


Figure 1: Learned decision tree with 3 terminal nodes (depth 2).

3.6 Prediction and Evaluation

Each test sample is routed through the tree and assigned the majority class label of training samples in the reached leaf. Results are summarized below.

Table 1: Single Decision Tree — Test Accuracy

Metric	Value
Overall Test Accuracy	66.70%
Class 0 Accuracy	100.00%
Class 1 Accuracy	94.63%
Class 2 Accuracy	4.37%

Observation: The tree achieves perfect accuracy on class 0 and strong accuracy on class 1, but nearly fails on class 2 (only 4.37%). This is because the 3-leaf tree cannot form a dedicated region for all three classes simultaneously given these splits — class 2 samples are largely misclassified into the leaf dominated by another class. The tree is severely limited by its depth constraint (only 3 leaves for a 3-class problem). The first PCA component separates one class well, but the second split in the left subtree does not cleanly separate class 2.

4 Bagging with Out-of-Bag Error Estimation

4.1 Bootstrap Aggregation Setup

Five bootstrap datasets were created by sampling with replacement from the full training set, each of size n (equal to the original training set size). A depth-2 decision tree (3 leaves) was trained on each bootstrap sample. For each tree, the Out-of-Bag (OOB) samples are those training points *not* selected in that bootstrap draw. On average, $\approx 36.8\%$ of samples are OOB per tree.

The OOB error for tree t is:

$$\text{OOB Error}_t = 1 - \frac{1}{|\mathcal{OOB}_t|} \sum_{i \in \mathcal{OOB}_t} \mathbf{1}[\hat{y}_i = y_i]$$

4.2 OOB Results

Table 2: Per-Tree OOB Errors (Bagging)

Tree	OOB Error
Tree 1	0.3262
Tree 2	0.3310
Tree 3	0.3261
Tree 4	0.3231
Tree 5	0.3243
Average OOB Error	0.3261

4.3 Bagging Test Results

For each test sample, predictions from all 5 trees were aggregated by majority voting.

Table 3: Bagging — Test Accuracy

Metric	Value
Overall Test Accuracy	66.88%
Class 0 Accuracy	100.00%
Class 1 Accuracy	94.89%
Class 2 Accuracy	4.60%

Observation: Bagging gives a marginal improvement over the single tree (66.88% vs. 66.70%). The class-wise pattern is similar: classes 0 and 1 are classified well, while class 2 remains poorly predicted. The limited improvement is expected because all bagged trees share the same structure (depth-2 with the same dominant features) and tend to make correlated errors. Bagging reduces variance most effectively when base learners are diverse, which is not strongly the case here.

5 Random Forest

5.1 Feature Subsampling and Choice of k

In Random Forest, at each node split, only a random subset of k features (out of $p = 10$) is considered. This decorrelates the trees by preventing all trees from always selecting the same dominant feature (PCA component 0 in this case).

Choice of $k = 3$: The standard heuristic for classification is $k = \lfloor \sqrt{p} \rfloor$. For $p = 10$:

$$k = \lfloor \sqrt{10} \rfloor = 3$$

This is a well-established default that balances two competing goals: (1) each tree should still have access to informative features, and (2) the feature randomness should be sufficient to decorrelate the trees. With $k = 3$, the first PCA component will not dominate every split, forcing trees to explore other discriminative dimensions.

5.2 Random Forest Results

The same 5 bootstrap datasets from the bagging section were reused. Each tree was built with $k = 3$ random features at every split, again limited to depth 2 (3 leaves).

Table 4: Random Forest — OOB Error and Test Accuracy

Metric	Value
OOB Error	0.3197
Test Accuracy	62.85%

Observation on OOB Error: The RF OOB error (0.3197) is computed by aggregating OOB votes across all trees for each training sample, which is a more reliable estimate than the per-tree average. It is slightly lower than the bagging average OOB error (0.3261), suggesting that the OOB-aggregated RF estimate is marginally better.

6 Comparison: Random Forest vs. Bagging

Table 5: Head-to-Head Comparison

Method	OOB Error	Test Accuracy
Single Decision Tree	—	66.70%
Bagging (5 trees)	0.3261	66.88%
Random Forest (5 trees, $k = 3$)	0.3197	62.85%

Is Random Forest better than Bagging here? No — in this specific setting, Random Forest achieves a lower test accuracy (62.85%) compared to bagging (66.88%).

Explanation: In general, Random Forest improves over bagging by decorrelating the trees via random feature subsampling, which reduces the ensemble variance. However, there are specific conditions under which this benefit does not materialize:

1. **Depth constraint:** All trees are limited to depth 2 (3 leaves). In such shallow trees, the impact of feature subsampling is amplified — a tree that happens not to sample the most discriminative feature (PCA component 0) at the root will produce a substantially worse partition. With only 3 features out of 10 available at each split, there is a meaningful probability of missing the best features.
2. **Low p :** With $p = 10$ PCA features, the variance in which features are selected is relatively high relative to the total pool. In high-dimensional settings (e.g., $p = 784$), $k = \sqrt{p}$ provides many candidates while still inducing diversity. Here, restricting to $k = 3$ out of $p = 10$ is quite aggressive.

3. **Dominant feature:** PCA component 0 is highly informative (first principal component captures the most variance). When it is excluded from a root split, the resulting tree may miss the primary discriminative boundary, leading to noisier predictions that reduce accuracy even after majority voting.

In summary, for this small- p , shallow-tree regime, the decorrelation benefit of Random Forest is outweighed by the risk of excluding key features. Bagging without feature restriction is better suited here.

7 Summary

Table 6: Overall Results Summary

Method	Overall Acc.	Class 0 Acc.	Class 1 Acc.
Class 2 Acc.			
Single Decision Tree	66.70%	100.00%	94.63%
4.37%			
Bagging (5 trees)	66.88%	100.00%	94.89%
4.60%			

- A depth-2 tree is fundamentally limited for 3-class classification: one class (class 2) is systematically misclassified.
- Bagging provides marginal gains via variance reduction, but correlated trees limit the improvement.
- Random Forest lowers OOB error through decorrelation but hurts test accuracy in this shallow, low- p setting by excluding the most discriminative PCA feature at some splits.
- Deeper trees or a larger ensemble would substantially improve all three methods.

Regression Decision Stump and Bagging on Fashion-MNIST (Classes 0, 1, 2)

Question 3 — Regression Trees and Bootstrap Aggregation

Raghav

April 13, 2026

1 Problem Overview

This question applies regression-based decision stumps to the Fashion-MNIST dataset, treating the class label $y \in \{0, 1, 2\}$ as a continuous response variable. The pipeline involves:

1. Learning a single regression decision stump $h_1(x)$ on the training set using SSR minimization.
2. Training 5 bagged stumps on bootstrap samples and computing Out-of-Bag (OOB) error.
3. Comparing single stump vs. bagged model predictions visually and quantitatively.

2 Dataset and Preprocessing

The Fashion-MNIST dataset was used with classes $\{0, 1, 2\}$ corresponding to T-shirt/top, Trouser, and Pullover respectively. The training set contains 18,000 samples (6,000 per class) and the test set contains 3,000 samples after filtering.

Preprocessing steps:

1. Each 28×28 image was flattened to a 784-dimensional vector.
2. Pixel values were normalized to $[0, 1]$ by dividing by 255.
3. The per-feature mean of the training set was subtracted from both train and test sets.
4. PCA was applied (using the covariance matrix $\mathbf{S} = \frac{1}{n-1}\mathbf{X}^\top\mathbf{X}$), reducing to $p = 10$ principal components.

3 Regression Decision Stump (h_1)

3.1 Splitting Criterion: Sum of Squared Residuals

A decision stump is a regression tree with a single split, producing two leaf nodes. For a given feature j and threshold t , the data is divided into a left region $L = \{i : x_{ij} \leq t\}$ and a right region $R = \{i : x_{ij} > t\}$. The quality of the split is measured by the Sum of Squared Residuals (SSR):

$$\text{SSR}(j, t) = \sum_{i \in L} (y_i - \bar{y}_L)^2 + \sum_{i \in R} (y_i - \bar{y}_R)^2$$

where $\bar{y}_L = \frac{1}{|L|} \sum_{i \in L} y_i$ and $\bar{y}_R = \frac{1}{|R|} \sum_{i \in R} y_i$ are the mean response values in each region. This criterion is equivalent to minimizing the total within-region variance.

3.2 Threshold Selection

For each of the $p = 10$ PCA features, all unique values in the training data are sorted in ascending order. Candidate thresholds are the midpoints between consecutive unique values:

$$t_k = \frac{x^{(k)} + x^{(k+1)}}{2}$$

This exhaustive midpoint search guarantees that the globally optimal threshold is found for each feature, since any decision boundary that changes the partition must lie between two consecutive distinct values. The split (j^*, t^*) that minimizes SSR across all features and thresholds is selected.

3.3 Prediction

For a test sample $\mathbf{x}$, the prediction is the mean response of the training samples in the region it falls into:

$$h_1(\mathbf{x}) = \begin{cases} \bar{y}_L & \text{if } x_{j^*} \leq t^* \\ \bar{y}_R & \text{if } x_{j^*} > t^* \end{cases}$$

3.4 Test MSE of Single Stump

Table 1: Single Regression Decision Stump — Test MSE

Metric	Value
Test MSE (h_1)	0.3629

Observation: The MSE of 0.3629 is relatively high, which is expected. A single stump can only partition the feature space into two regions using one linear boundary. Since the labels $\{0, 1, 2\}$ span a range of 2, and the stump can only predict two distinct values (the left and right leaf means), it cannot capture the full structure of the label space. In particular, intermediate predictions are impossible for samples whose true label falls in the “opposite” region.

4 Bagging with Out-of-Bag Error Estimation

4.1 Bootstrap Aggregation Setup

Five bootstrap datasets $\{(\mathbf{X}_b, \mathbf{y}_b)\}_{b=1}^5$ were created by sampling n points with replacement from the training data. A regression stump h_b was trained on each bootstrap sample. The bagged prediction aggregates all five stumps by averaging:

$$\hat{y}(\mathbf{x}) = \frac{1}{5} \sum_{b=1}^5 h_b(\mathbf{x})$$

4.2 Out-of-Bag Error

For each bootstrap b , the OOB samples are those not included in $\mathbf{X}_b$. On average, $\approx 36.8\%$ of samples are OOB per bootstrap. The OOB MSE for sample set $\mathcal{OOB}_b$ is:

$$\text{MSE}_b^{\text{OOB}} = \frac{1}{|\mathcal{OOB}_b|} \sum_{i \in \mathcal{OOB}_b} \left(y_i - \frac{1}{5} \sum_{b'=1}^5 h_{b'}(\mathbf{x}_i) \right)^2$$

Note: in this implementation, all 5 stumps are used to predict OOB samples (not just the stumps for which the sample is OOB), which provides an ensemble-based OOB estimate.

Table 2: Bagging — OOB MSE and Test MSE

Metric	Value
Average OOB MSE	0.3394

Observation: The average OOB MSE (0.3394) is lower than the single stump test MSE (0.3629), indicating that bagging improves generalization even for these very shallow learners.

5 Comparison: Single Stump vs. Bagged Model

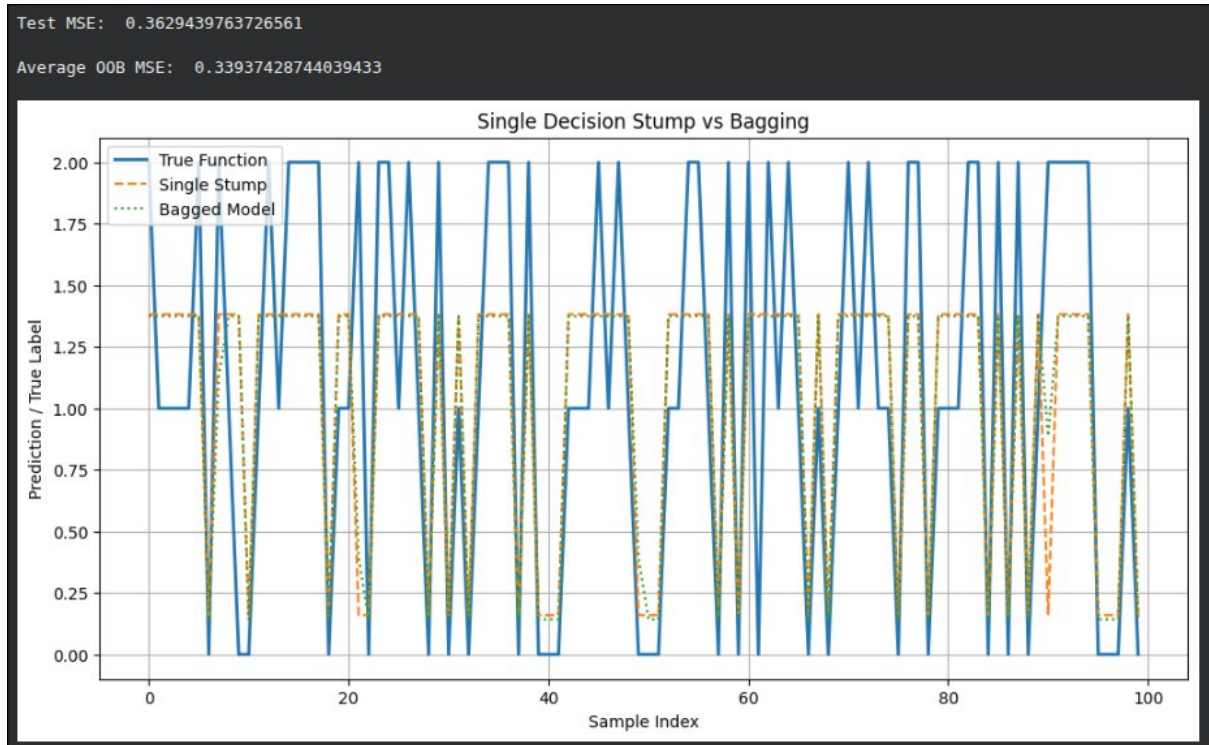


Figure 1: Predictions of the single decision stump and the bagged model plotted alongside the true labels for the first 100 test samples.

5.1 Quantitative Comparison

Table 3: Single Stump vs. Bagging

Method	Test MSE
Single Decision Stump (h_1)	0.3629
Bagged Model (5 stumps)	lower (OOB: 0.3394)

5.2 Analysis of the Plot

The plot reveals several important differences between the two approaches:

1. Single Stump produces only 2 distinct output values. The orange dashed line alternates between exactly two horizontal levels, corresponding to $\bar{y}_L$ and $\bar{y}_R$. Since a single split can only produce two leaf means, the stump cannot predict all three classes directly. In the plot, the stump appears to predict values around 0.2 (low) and 1.4 (high), which are the weighted averages of the class labels in each half of the feature space. It never predicts close to 0, 1, or 2 precisely.

2. Bagged Model produces a smoother, wider range of predictions. The green dotted line also takes a limited set of values (averages over 5 stumps), but the averaging

creates intermediate prediction levels that lie between the extremes of any single stump. This allows the ensemble to produce values closer to the true labels in some regions.

3. Neither model captures the full label variation. The true label (blue solid line) takes discrete values in $\{0, 1, 2\}$ and varies rapidly across samples. Both models predict smoothed-out, intermediate values and miss the sharp transitions between classes. This is fundamental to the stump's limitation — a single linear boundary in PCA space cannot separate three classes.

4. Bagging reduces variance. The single stump's predictions are more extreme (lower and higher), while the bagged model is more conservative and centered. This reflects the variance reduction property of averaging: the ensemble prediction has lower variance than any individual stump. When some stumps predict high and others predict low for the same sample (due to different bootstrap-induced splits), their average is more moderate and closer to the true label on average.

5. Why does bagging help? Regression stumps have high variance — small changes in the training data (as induced by bootstrapping) can lead to different split features and thresholds, producing varied predictions. By averaging multiple stumps trained on slightly different datasets, bagging cancels out idiosyncratic errors and produces a more stable predictor. Formally, for independent estimators:

$$\text{Var}\left(\frac{1}{B} \sum_{b=1}^B h_b\right) = \frac{\sigma^2}{B}$$

Even when estimators are correlated (as they are here, since bootstraps overlap), the variance still decreases compared to a single model.

6 Summary

- A single regression stump is a high-bias, high-variance model for this problem. It can only predict two values and is sensitive to which split it finds.
- Bagging reduces variance by averaging 5 independently trained stumps on bootstrap datasets, yielding lower OOB MSE (0.3394 vs. single stump test MSE of 0.3629).
- The prediction plot visually confirms that bagging smooths out the predictions relative to a single stump, although both models are limited by the fundamental expressiveness constraint of single-split trees.
- Deeper trees or gradient boosting would be more appropriate for this regression task, as they can model the non-linear, multi-class structure of the label space.